ANDROID REMOTE CONTROL

Master → Many Hosts

Requirements, architecture, build plan, agent prompts, and security/ops for a Kotlin Master/Host app with Magisk on Host and an always-on relay server.

VERSION

1.0

DATE

August 2026

STACK

Kotlin + Magisk Host + relay server

SCOPE

Sideload APKs · dynamic Hosts · dynamic access

Remote Control — Documentation Index

Android project: **1 Master → many Hosts** remote screen share and optional remote control (different networks / cities). Master can **unlock the Host device**, stream audio, stop/lock apps (per flags). Custom Kotlin app + Magisk on Hosts. Not a Skitii clinical product.

Docs in this folder

Doc	Purpose
01-requirements.md	What to build (goals, features, non-goals, acceptance criteria)
02-architecture.md	Master / Host / Magisk / server, data model, networking
03-build-plan.md	Phased how-to-build and test order
04-agent-prompts.md	Copy-paste prompts for other AI agents
05-security-and-ops.md	Auth, Magisk risk, AWS cost, lock screen, ops

Quick summary

- **Master phone:** normal Android app (no Magisk). Lists Hosts, connects, views screen, can **unlock Host device**, send taps, stop/lock apps, play Host audio (per flags).
- **Host phones:** Magisk + Host app. Silent screen capture + input inject + unlock/audio/stop/lock. Dynamic add/remove.
- **Access:** per-Host and dynamic (`VIEW_ONLY` vs `FULL_CONTROL` + flags: `audio`, `remoteUnlock`, `stopApps`, `appLock`).
- **v1 includes:** system audio, **Master unlocks locked Host**, stop apps, and app lock (when flags enabled).
- **Network:** small always-on **relay** server (e.g. AWS Lightsail ~$5–10/mo). ~20 min/day usage. WebRTC optional later.
- **Stack:** Kotlin, Android Studio, physical device testing. Sideload APKs (no Play Store for v1).

How to use with agents

1. Give the agent 01-requirements.md + 02-architecture.md as context.
2. Pick a phase from 03-build-plan.md.
3. Paste the matching prompt from 04-agent-prompts.md.
4. Keep 05-security-and-ops.md in scope for any auth/network work.

Out of scope reminders

- Do not rebuild Magisk.
- Do not use Device Owner / MDM (not needed with Magisk full control).
- Do not use React Native for Host root core.

- Do not build WebRTC/TURN in v1 unless asked (relay is enough).
- No Multi-Master.
- Host display names are labels only — access profiles are not hard-coded to specific devices.

01 — Requirements

Goal

Build a **custom Android remote assistance app** so one **Master** phone can connect to **many Host** phones over the internet (different cities), with:

- Screen share (see Host screen)
- Optional remote control (tap / scroll) when access allows
- **Master can unlock the Host device** (lock screen) when `remoteUnlock` is enabled
- Minimal Host-side interaction after one-time setup (Magisk path = no per-session Allow dialog)
- Dynamic Hosts and dynamic access (not hard-coded device lists)

Owner installs and configures each Host. Master connects when needed (~20 minutes/day typical).

Roles

Role	Device	Magisk	Responsibility
Master	Controller phone	Not required	Login, Host list, connect, view, control, **unlock device**, stop/lock apps, hear audio (per flags)
Host	Controlled phone(s)	Required for silent capture + inject	Run background service, stream, apply access policy

Devices are authorized by the account owner. Not a public Play Store product for v1.

Core features (must)

1. **Custom app** (Kotlin) — Master + Host modes (flavors or in-app role). Not AnyDesk/TeamViewer.
2. **1 Master → many Hosts** — add / remove / rename Hosts anytime.
3. **Dynamic access** per Host (server-backed), e.g.:
   - `screenShare`
   - `remoteControl`
   - `stopApps` (v1 must when enabled — e.g. force-stop YouTube / packages)
   - `appLock` (v1 must when enabled — block or lock selected apps)
   - `audio` (v1 must when enabled for that Host)
   - `remoteUnlock` (v1 must when enabled for that Host — high risk)
4. **Internet** — works across cities (not same Wi-Fi only). Same Wi-Fi allowed for early testing.
5. **Host online presence** — heartbeat; Master sees online/offline.
6. **Pairing** — code or QR; Host linked to Master account only.
7. **Auth** — Master login; sessions rejected without valid token.

8. **Magisk on Host** — root grant only to Host app; silent screen capture + input inject when policy allows.
9. **Foreground service** on Host + start-on-boot (or clear restart instructions).
10. **Connect-on-demand streaming** — capture/stream starts when Master connects (e.g. FCM wake); stops after disconnect. Cheap always-on server stays up (do not start/stop VM every session for v1).
11. **System audio stream** — Host app/system sound (e.g. YouTube) playable on Master when `audio` is enabled.
12. **Master unlocks Host device** — When Host screen is locked and `remoteUnlock` is enabled, Master taps Unlock; Host wakes and unlocks lock screen (PIN/password/pattern via Magisk/root). Credential stored only on Host, encrypted; never in Master APK plaintext. After unlock, Master can view/control/audio as other flags allow.

Example access profiles (not hard-coded)

Example profile	Intended access
Host A	`FULL_CONTROL`: screen + remote control + audio + remote unlock + stop apps + app lock
Host B	`VIEW_ONLY`: screen share only (no control, unlock, stopApps, appLock, or required audio)
Host C	`VIEW_ONLY`: screen share only

Profiles are settings on each Host record. Any new phone can be added with any profile.

With Magisk **FULL_CONTROL**, Master can already open Settings, lock/stop apps, uninstall apps, etc. via remote UI — **Device Owner / MDM is not required**. `stopApps` and `appLock` are first-class Host features (not "optional later").

Later / optional (not v1)

- **WebRTC + TURN** — lower latency than server relay. Not required for v1; relay is enough for ~20 min/day. (WebRTC = phones try direct media; TURN = fallback relay when direct fails.)
- Multi-Master — out of scope

Non-goals

- Rebuilding Magisk or a Magisk clone
- Play Store listing for v1
- iOS
- Zero-server P2P only (unreliable across cities)
- Unauthenticated open Hosts on the internet
- React Native / Flutter for Host root capture and inject core

- Perfect commercial AnyDesk parity in v1
- Device Owner / MDM lockdown (not needed; Magisk full control covers app lock / uninstall / settings)
- Multi-Master
- WebRTC + TURN in v1 (optional later only)

Lock screen behavior

State	Expected
Host unlocked	Full screen share (+ control / audio if allowed)
Host locked, `remoteUnlock=false`	Lock screen or black; Master may wake screen only
Host locked, `remoteUnlock=true`	Master can unlock (PIN/password/pattern credential configured on Host) then use session

Tech constraints

Area	Choice
Language	Kotlin
UI	Jetpack Compose (preferred)
IDE	Android Studio required to build/run; Cursor optional for editing
Testing	Physical phones (Magisk Host + Master). Emulator insufficient for root path
Install	Sideload APK via adb
Backend	Small always-on VPS (e.g. AWS Lightsail ~$5–10/mo)
Usage budget	~20 min/day → roughly $5–15/mo AWS all-in

Acceptance criteria (v1)

- [ ] Master can add a Host via code/QR and see it in a list
- [ ] Master sees Host online/offline
- [ ] Master connects over internet and sees Host screen (unlocked Host)
- [ ] When `remoteControl=true`, Master tap/scroll controls Host
- [ ] When `remoteControl=false`, Master cannot control Host (view only)
- [ ] When `audio=true`, Master hears Host system/app audio (e.g. YouTube)
- [ ] When `remoteUnlock=true`, **Master can unlock the locked Host device** and then control/view
- [ ] When `stopApps=true`, Master can force-stop selected packages on Host
- [ ] When `appLock=true`, Master can lock/block selected apps on Host

- [ ] Access flags changeable without reinstalling Host app
- [ ] Unauthenticated connect rejected
- [ ] Host can be removed from Master account
- [ ] Magisk root used only on Host; Master has no Magisk requirement
- [ ] No per-session MediaProjection Allow on Magisk Host path

Glossary

- **Master** — controller / viewer phone
- **Host** — controlled / shared phone
- **Magisk** — free open-source root tool (official GitHub only); not a Play Store app
- **Silent control** — no system Allow dialog each session (requires root on Host)

02 — Architecture

High-level

```
Master app (no Magisk)
        |
        |  HTTPS + WSS (auth)
        ▼
┌──────────────────────────┐
|  Backend (always-on)     |
|  - REST API              |
|  - WebSocket signaling   |
|  - Media/input relay (v1) |
|  - DB: users, hosts      |
└──────────────────────────┘
        |
        |  HTTPS + WSS
        ▼
Host app (Magisk root)
  - Heartbeat / online
  - Screen capture (root)
  - Input inject (root) if policy allows
  - Applies access flags from server
```

Real use: Host and Master on **any** networks (home Wi-Fi, mobile data, different cities).
Early test: same Wi-Fi is fine; still prefer going through the server so the internet path is ready.

App structure

Single Android project with two flavors (recommended):

Flavor	Package / applicationId suffix	Magisk
`master`	`.master`	No
`host`	`.host`	Yes

Alternative: one APK with “This device is Master / Host” — flavors are cleaner for permissions and Play-like separation later.

Master responsibilities

- Login / session token
- List Hosts (name, id, online, access summary)
- Add Host (code/QR), rename, remove, edit access
- Connect → show video surface

- **Unlock Host device** when locked and `remoteUnlock` is enabled
- Map touch to Host coordinates when `remoteControl` allowed
- stopApps / appLock / audio controls when those flags allowed
- Disconnect / idle timeout

Host responsibilities

- Pair to Master account
- Request Magisk root once; refuse to stream without root (v1 Magisk path)
- Foreground service + boot receiver (or documented restart)
- Heartbeat to server
- On connect (push or WS command): start capture + send frames
- Apply input only if server policy says `remoteControl`
- On disconnect: stop capture (keep lightweight heartbeat)

Magisk vs app

```
┌──────────────────────────────┐
│  Your Host APK (Kotlin)      │  ← you build this
├──────────────────────────────┤
│  Android sandbox             │
├──────────────────────────────┤
│  Magisk (boot / root)        │  ← existing tool; do not rebuild
└──────────────────────────────┘
```

Magisk unlocks root. Your app uses root (e.g. libsu) for:

- Screen frames without MediaProjection Allow dialog
- Touch/scroll inject without Accessibility prompts

Master never needs Magisk.

Access model

Server is source of truth. Host caches last policy and refreshes on connect/heartbeat.

```
{
  "hostId": "h_abc123",
  "name": "Host-A",
  "access": {
    "screenShare": true,
    "remoteControl": true,
    "stopApps": true,
    "appLock": true,
    "audio": true,
```

```
    "remoteUnlock": true
  }
}
```

Convenience presets (UI only):

- `VIEW_ONLY` → screenShare true; remoteControl / audio / remoteUnlock false
- `FULL_CONTROL` → screenShare + remoteControl true; audio / remoteUnlock / stopApps / appLock as chosen (all are v1 features, not deferred)

Data model (minimal)

User (Master account)

- `id`, `email` or username, `passwordHash`, `createdAt`

Host

- `id`, `ownerUserId`, `name`, `pairingSecret` / device token
- `access` (JSON flags)
- `fcmToken` (optional)
- `lastSeenAt`, `online`
- `createdAt`
- Unlock credential: **Host-local only** (Android Keystore / encrypted prefs). Server stores `remoteUnlock` flag, not the PIN.

Session (optional table)

- `id`, `hostId`, `masterUserId`, `startedAt`, `endedAt`, `status`

Networking

v1 (required): relay through server

- Host uploads frames (+ audio if enabled) → server → Master
- Master sends input / unlock commands → server → Host
- Simpler than WebRTC; more bandwidth on VPS
- Fine for ~20 min/day at 480p / low fps

Later (optional): WebRTC + TURN

Not in v1. For lower latency later:

- **WebRTC** — Master and Host try a direct media path (like a call)
- **TURN** — fallback relay when firewalls/NAT block direct path

v1 server relay is enough until latency becomes a problem.

Discovery

- No LAN-only discovery for production

- Hosts register to account; Master lists by `ownerUserId`

Push wake

- Master taps Connect → server sends FCM to Host → Host starts stream service
- Avoids 24/7 heavy capture

Screen + input + audio pipeline (Host)

1. Root capture → bitmap/frames (target 5–10 fps, ~480p for v1)
2. Encode JPEG (v1) or H.264 (later)
3. If `audio`: capture system/internal audio (root); encode Opus/AAC; mux or parallel stream
4. Send over WSS binary (preferred) via server relay
5. Receive input events `{ type: tap|swipe, x, y, ... }`
6. Scale coordinates to Host display size
7. Inject via root (`input` / uinput) only if `remoteControl`
8. If `remoteUnlock`: accept unlock command; inject credential via root only when flag true

Lock screen

- Unlocked: normal stream (+ audio if enabled)
- Locked + `remoteUnlock=false`: lock UI or black; optional `wakeScreen` only
- Locked + `remoteUnlock=true`: Master sends unlock → Host unlocks with stored credential → then full session

Server deployment sketch

Component	Suggestion
Runtime	Node (Nest/Express) or Go — team preference
Process	systemd / Docker on Lightsail
TLS	HTTPS + WSS (Caddy/nginx or cloud cert)
DB	SQLite (tiny) or Postgres
Cost	Always-on ~$5–10/mo; billing alarm at $5/$10

Do **not** design v1 around starting/stopping the VM from the APK (cold start + secret handling). Keep server always on; start **stream** on demand.

Security boundaries

- TLS everywhere
- Host accepts commands only with valid device token for that Host
- Master only sees Hosts for `ownerUserId`
- Magisk grant only to Host app

- Unlock PIN/pattern stored **on Host only**, encrypted; enable `remoteUnlock` only on Hosts that need it
- Compromised Master account ≈ unlock + full control on those Hosts — strong Master auth required
- No Device Owner required: Magisk full control covers settings / app lock / uninstall via remote UI

03 — Build Plan

Work in phases. Each phase has a clear “done” check. Prefer physical phones; same Wi-Fi for early streaming tests; internet before calling the project usable.

Phase 0 — Tooling

Do

1. Install Android Studio (SDK, platform tools). adb already OK if present.
2. Create empty Kotlin + Compose project.
3. Add product flavors: `master`, `host`.
4. Create Git repo. Sideload with `adb install -r`.

Done when: both flavors install and open on two physical phones.

Phase 1 — App shells + Magisk gate

Host

- UI: Host name, Host ID placeholder, status “Waiting”
- Foreground service skeleton
- Magisk/root request (libsu or equivalent); show granted/denied

Master

- Simple login UI (local or against stub API)
- Empty Host list
- Connect button (no-op)

Done when: Host obtains root grant; Master opens list screen.

Phase 2 — Backend basics

Server

- Register/login Master
- Register/pair Host
- List Hosts for owner
- Host heartbeat → `online` / `lastSeenAt`
- WebSocket connect with auth token

Done when: Host appears **online** on Master without video.

Phase 3 — Dynamic Hosts + access

1. Host shows pairing code or QR.
2. Master “Add Host” → pair.

3. Master rename / remove Host.
4. Master edits access flags; Host fetches policy.

Done when: add two Hosts; one VIEW_ONLY, one FULL_CONTROL (with audio/remoteUnlock as needed); flags persist after app restart.

Phase 4 — Screen share (same Wi-Fi via server)

1. Host root capture frames.
2. Relay through server (or temporary direct only for debug — prefer server).
3. Master renders frames.
4. Target: ~480p, 5–10 fps, JPEG OK.

Done when: Master sees unlocked Host screen live for several minutes.

Phase 5 — Remote control + stopApps + appLock

1. Master maps touches to normalized or pixel coordinates.
2. Host injects tap/scroll if `remoteControl=true`.
3. Host **ignores** input if `remoteControl=false`.
4. `stopApps` (v1 must): Master sends package name(s); Host force-stops when flag true.
5. `appLock` (v1 must): Master configures locked/blocked package list; Host enforces when flag true (block launch / re-lock as designed).

Done when: FULL_CONTROL Host is controllable; VIEW_ONLY Host is not; stopApps and appLock work when flags are on.

Phase 6 — Internet + on-demand stream

1. Point apps at public HTTPS/WSS URL.
2. Test Master on mobile data, Host on home Wi-Fi.
3. FCM (or equivalent): Connect → wake Host → start capture.
4. Disconnect → stop capture; heartbeat remains.

Done when: cross-network session works ~20 minutes; reconnect works.

Phase 7 — System audio (v1 must)

1. Host: root capture of system/internal audio (document approach).
2. Stream with video via server relay.
3. Master: play audio; basic A/V sync tolerance.
4. Enforce `access.audio` — no audio path when false.
5. Do not silently fall back to microphone-only without labeling it.

Done when: YouTube (or similar) on Host is audible on Master with `audio=true`.

Phase 8 — Master unlocks Host device (v1 must)

1. Host setup: store unlock credential in Android Keystore / encrypted storage (PIN/password/pattern as supported).
2. Access flag `remoteUnlock` (enable on Hosts where Master must unlock the device).
3. Master UI: clear **Unlock device** action when Host reports locked.
4. Host: wake display + inject unlock via root only if flag true.
5. After unlock, normal screen/control/audio/stopApps/appLock session continues.
6. Document risk: stolen Master session can unlock Host device.

Done when: Master can unlock a locked Host (flag on) and then use the device without anyone tapping the Host.

Phase 9 — Harden

- TLS, strong passwords, session timeout
- Live-session indicator on Host
- AWS billing alarm
- Document Magisk install for each Host model
- Confirm Device Owner not used; app lock/uninstall via Magisk control when needed

Suggested calendar

Week	Focus
1	Phase 0–1
2	Phase 2–3
3–4	Phase 4
5	Phase 5
6	Phase 6
7	Phase 7–8
8	Phase 9

Test matrix

Test	Network	Host state	Access	Expect
Online list	any	app running	—	online badge
View	same Wi-Fi	unlocked	VIEW_ONLY	video; no inject
Control	same Wi-Fi	unlocked	FULL_CONTROL	taps work
Stop apps	any	unlocked	stopApps=true	package force-stopped
App lock	any	unlocked	appLock=true	locked apps cannot be used as designed
Audio	any	unlocked	audio=true	Master hears Host app audio
Unlock	any	locked	remoteUnlock=true	**Master unlocks Host device**; then session works
Locked denied	any	locked	remoteUnlock=false	lock/black; no unlock
Internet	different nets	unlocked	FULL_CONTROL	video + taps (+ audio if on)
Auth fail	any	—	—	rejected

First milestone (celebrate)

Master sees one Host online → Connect → **live screen** (same Wi-Fi via server).

After that milestone: add control, then internet, then **audio** and **remote unlock**. Do not start WebRTC before those are stable.

04 — Agent Prompts

Copy a prompt below into another AI agent (Cursor, etc.). Always attach or point to:

- `content/remote-control/01-requirements.md`
- `content/remote-control/02-architecture.md`
- Relevant phase in `03-build-plan.md`
- `05-security-and-ops.md` for auth/network work

Global rules for every agent

- Kotlin Android app: Master (no Magisk) + Host (Magisk).
- Do not rebuild Magisk. Do not use Device Owner for v1.
- Do not use React Native for Host capture/inject.
- Hosts and access are dynamic (server-backed). Do not hard-code Host names or access maps.
- v1 includes system audio, remote unlock, stopApps, and appLock (feature flags). No Device Owner. No WebRTC in v1 unless asked.
- Prefer small, phase-scoped PRs. Physical device testing.
- Never commit secrets, Host PINs, or AWS keys in the APK. Unlock credentials stay on Host only.

Prompt A — Scaffold Android project

```
Read remote-control docs 01-requirements and 02-architecture.

Create a new Kotlin Android project (Compose) with product flavors:
- master (applicationId suffix .master)
- host (applicationId suffix .host)

Add:
- Minimal Master screen: login placeholder, empty host list, Connect button
- Minimal Host screen: status Waiting, Host name field
- Host foreground service skeleton + notification channel
- Shared module or common package for API models (Host, AccessFlags)

Do not implement Magisk, video, or backend yet.
Follow Phase 0–1 of 03-build-plan.
```

Prompt B — Magisk / root gate on Host

```
Read remote-control 01, 02, and 05 (security).
```

```
On the Host flavor only:
- Integrate a maintained root library (e.g. libsu)
- On start, request root; show Granted / Denied in UI
- Gate streaming APIs behind root granted
- Document that Magisk must be installed from official GitHub only

Master flavor must not request root.
No screen capture implementation yet beyond a stub function that checks root.
```

Prompt C — Backend API + WebSocket

```
Read remote-control 01–02 and Phase 2 of 03-build-plan.

Build a small backend (Node/Nest or Express is fine) with:
- Master register/login → JWT
- Host register/pair with one-time code
- GET /hosts for owner (id, name, online, access flags)
- PATCH /hosts/:id access flags and name
- DELETE /hosts/:id
- Host heartbeat POST every 15–30s
- Authenticated WebSocket for signaling

Use Postgres or SQLite. TLS assumed at reverse proxy.
Provide .env.example without real secrets.
No video relay binary protocol yet — text signaling events only:
host.online, session.request, session.accept, session.end
```

Prompt D — Pairing + dynamic access UI

```
Read remote-control Phase 3.

Implement:
- Host displays rotating or static pairing code
- Master Add Host flow
- Master Host detail: rename, remove, toggle access flags
  (screenShare, remoteControl, stopApps, appLock, audio, remoteUnlock)
- Presets VIEW_ONLY and FULL_CONTROL that set flags
- Host polls or receives WS update for policy; caches locally

Enforce: Master UI hides control / unlock / audio chrome when those flags are false.
```

Prompt E — Root screen capture + Master viewer

```
Read remote-control Phase 4 and architecture screen pipeline.

Implement v1 screen share:
- Host: root-based frame capture (document approach used)
- Encode JPEG ~480p, 5–10 fps
- Send frames over authenticated WebSocket via server relay
- Master: render latest frame on a full-screen surface

Constraints:
- Magisk Host path only (no MediaProjection Allow dialog requirement)
- Same Wi-Fi or internet both OK if using public server URL
- Stop capture on session end

Do not implement touch inject in this task.
```

Prompt F — Remote control + policy enforcement

```
Read remote-control Phase 5.

Implement:
- Master: touch on video → send tap/swipe events with coordinates
- Scale correctly for Host resolution / letterboxing
- Host: inject via root only if access.remoteControl == true
- If false, ignore input and log
- stopApps / appLock: implement in Prompt F2 (v1 must) — do not skip as optional

Add tests or manual test checklist for VIEW_ONLY vs FULL_CONTROL.
```

Prompt F2 — stopApps + appLock (v1 must)

```
Read remote-control Phase 5 and requirements (stopApps and appLock are v1 must, not optional).

Implement:
- stopApps: Master UI to force-stop packages; Host executes via root when access.stopApps == true
- appLock: Master configures package block/lock list; Host enforces when access.appLock == true
- Ignore both when flags are false
- Persist lock list on Host; sync from server policy when possible

Add manual test checklist for both flags on/off.
```

Prompt G — FCM wake + internet hardening

```
Read remote-control Phase 6 and 05-security-and-ops.

Implement on-demand streaming:
- Master Connect → server → FCM data message to Host
- Host wakes service, starts capture, joins session
- Disconnect stops capture; heartbeat continues
- Point apps at production HTTPS/WSS base URL via BuildConfig

Keep AWS/Lightsail server always on. Do NOT start/stop EC2 from the APK.
Add idle session timeout (e.g. 30 min).
```

Prompt H — System audio (v1)

```
Read remote-control Phase 7 and requirements (audio is v1 must).

Implement:
- Host: capture system/internal audio with root where possible
- Stream alongside video via server relay
- Master: play audio with basic A/V sync tolerance
- Feature flag access.audio must be true

If platform limitations block clean app audio, document limitation; do not fake with microphone-
only without labeling it.
```

Prompt H2 — Master unlocks Host device (v1)

```
Read remote-control Phase 8 and 05-security-and-ops (remote unlock).

Requirement: Master can unlock the Host device when the screen is locked.

Implement:
- Host setup UI to store unlock credential in Android Keystore / encrypted storage
- access.remoteUnlock flag from server
- Master "Unlock device" action when Host is locked
- Host: wake + root inject unlock only if flag true
- Never send raw PIN to server logs; never store PIN in Master APK

Document security risk if Master account is compromised.
```

Prompt I — Docs / handoff update

```
Update remote-control docs to match what was actually implemented
(APIs, package names, how to run server, adb install commands).
Keep agent prompts in sync. Do not invent features not in the repo.
```

Prompt J — Full context bootstrap (new chat)

```
You are building a Master→many Hosts Android remote-control system.

Read all docs under content/remote-control/ in Skitii-Docs-Generation:
README, 01-requirements, 02-architecture, 03-build-plan, 04-agent-prompts, 05-security-and-ops.

Summary constraints:
- Kotlin Master + Host flavors
- Magisk on Host only for silent screen + input
- Dynamic hosts and dynamic access flags
- v1 must include system audio, remote unlock, stopApps, and appLock (per-Host flags)
- No Device Owner / MDM; Magisk full control is enough for app lock/uninstall/settings
- No WebRTC/TURN in v1 (server relay only); no Multi-Master
- Small always-on relay server; ~20 min/day usage
- Physical device testing; sideload APKs
- No Magisk clone, no RN Host core

Ask which phase to implement next, then implement only that phase.
```

05 — Security and Operations

Magisk

- Magisk is **free** and open source. Install only from the **official GitHub** releases.
- Magisk is **not** a Play Store app. Fake “Magisk” APKs are often malware.
- Magisk alone does **not** put the phone on the open internet for remote control.
- Risk rises when: fake tools, root granted to unknown apps, or a remote Host service with weak/no auth.

Host policy

- Grant Magisk root **only** to your Host app.
- Master phone: **no Magisk**.

Auth and remote risk

Anyone who can authenticate as Master (or talk to an open Host) can see and possibly control that phone.

Required

- Strong Master password or equivalent
- TLS (HTTPS / WSS)
- Host device token; Host only accepts commands for its own id
- Master only lists Hosts owned by that account
- Reject unauthenticated WebSocket/API calls

Recommended

- Session timeout / idle disconnect (~20–30 min matches usage)
- Live-session indicator on Host UI/notification
- Ability to revoke a Host from Master (delete + invalidate token)
- Billing / spend alarm on cloud account

Avoid in v1

- Putting long-lived AWS access keys inside the APK
- Starting/stopping cloud VMs from the client (complex + secret sprawl)
- Storing unlock PIN on the **server** or in Master app plaintext

Lock screen

Scenario	Behavior
Unlocked Host	Screen share (+ control / audio if allowed) works
Locked, `remoteUnlock=false`	Lock screen or black; optional wake only
Locked, `remoteUnlock=true`	**Master unlocks the Host device** via Host-local credential + root inject

Remote unlock security

- Credential lives **on Host only** (Keystore / encrypted prefs)
- Flag `remoteUnlock` is per-Host; default off for view-only Hosts
- Strong Master auth required — compromise unlocks those Hosts
- Prefer enabling unlock only on Hosts that need unattended access

Access profiles (ops)

Configure per Host on the server/Master UI — examples only:

Profile	Flags
View only	screenShare on; remoteControl / audio / remoteUnlock off
Full control	screenShare + remoteControl; audio / remoteUnlock / stopApps / appLock as needed (all v1)

Device Owner / MDM is **not** required. Magisk full control can lock apps, change settings, uninstall, etc. via the remote session.

Changing flags should not require reinstalling the Host APK.

AWS / server cost (guidance)

Usage assumption: **~20 minutes/day** screen share.

Item	Rough monthly
Lightsail / small always-on VM	$5–10
Egress for ~10 hours/month low-bitrate video	~$1–3
Total ballpark	**$5–15**

Tips:

- Prefer 480p / low fps for assistance sessions
- Keep server always on; start **stream** on Connect only
- Set billing alerts at $5 and $10

Free tiers exist for experiments; video relay can exceed free limits — fine for learning, not a promise for daily production use.

Testing ops

Phase	Network	Notes
Early video	Same Wi-Fi OK	Still use server URL if possible
Production-like	Different networks / mobile data	Required before “done”
Install	`adb install -r`	USB debugging on devices
Magisk Host	Physical device only	Emulator not a substitute

ADB is for **developer install/debug only**. Day-to-day use does not need adb or same Wi-Fi.

Incident basics

If a Master password may be leaked:

1. Change Master password / rotate tokens server-side
2. Remove or re-pair all Hosts
3. Revoke FCM/device tokens
4. Review Magisk: ensure only Host app has root

If a Host phone is lost:

1. Delete Host from account (server revoke)
2. If possible, remote wipe via Google account
3. Assume device may be unlocked by finder if no strong screen lock

Consent and transparency

- Screen share and control are sensitive. Prefer **view-only** unless control is needed.
- Inform Host users when view or control access is enabled.
- Show a clear “Session active” state on Host when Master is connected.